

RESTAURARTE

Sistema de Gerenciamento de Restaurante em Terminal

Apresentação do Trabalho A3

Obely Rodrigues de Pontes

Daniel Camargos da Cunha

Ian Fernandes Guimarães

Miguel Dolabella Miranda

Pedro Henrique Penha Piantino

Algoritmos e Programação FASEH 2026

Tema do Projeto

O que é o RestaurARTE?

O **RestaurARTE** é um sistema completo de gerenciamento de restaurante desenvolvido inteiramente em **Java 21**, executado no terminal do computador. O sistema simula o ciclo real de atendimento de um restaurante do cadastro do cliente ao pagamento da conta.

Ficha Técnica do Projeto

- Linguagem: Java 21
- Repositório: github.com/Meinards15/trabalho-algoritimo
- Banco de dados: SQLite

Tipo de Sistema	TUI Terminal User Interface (sem interface gráfica)
Persistência	Banco de dados SQLite local (arquivo data/restaurarte.db)
Arquitetura	4 camadas: Interface (TUI) Serviços Modelos Banco
Funcionalidades	Cadastro, login, pedidos, cardápio, avaliações, histórico
Build	Gradle 9.3 com fat-jar standalone ./gradlew build

Problema Resolvido

Por que o RestaurARTE existe?

Pequenos e médios restaurantes frequentemente gerenciam suas operações de forma manual ou com ferramentas genéricas que não se adaptam ao fluxo real do negócio. Identificamos **cinco problemas centrais**:

Problemas Reais

- ✗ Controle de pedidos em papel sujeito a erros e perda.
- ✗ Sem histórico digital de clientes e visitas.
- ✗ Sem mecanismo de coleta de avaliações.
- ✗ Dados perdidos ao fechar o programa.
- ✗ Login complexo com senha: barreira desnecessária.

Nossa Solução

- Pedidos digitais com abertura e fechamento registrados.
- Perfil persistente: histórico completo de cada cliente.
- Sistema de avaliações com nota 0-5 e média automática.
- Banco SQLite: dados salvos permanentemente.
- Acesso apenas pelo nome simples e rápido.

Demonstração dos Menus

O programa em funcionamento Parte 1

O programa é inteiramente operado pelo teclado. O usuário vê menus numerados e digita o número da opção desejada. Abaixo, os três primeiros menus do fluxo principal:

1. Tela de Inicialização - Banner ASCII e Menu Principal

Tela inicial

2. Tela de Acesso - Login e Cadastro Unificado pelo Nome

Tela de acesso

Demonstração dos Menus

O programa em funcionamento Parte 2

3. Perfil do Cliente - exibido após login bem-sucedido

Perfil ex

4. CardÆpio - seleção de pratos por ID

CardÆpio

5. Pedido Atual - resumo com total calculado

Lista dos pr

Demonstração dos Menus

O programa em funcionamento - Parte 3

6. Sistema de Avaliações - nota de 0 a 5 com comentário

Tela de avaliação

7. Listagem de Avaliações - visão pública com média geral

Lista de todas as avaliações

8. Encerramento do Pedido - confirmação e registro de pagamento

Tela de confirmação

Estrutura das Classes

Arquitetura em 4 camadas

O projeto é organizado em **quatro camadas** com responsabilidades rigorosamente separadas. Nenhuma camada ultrapassa os limites da outra:

CAMADA	CLASSES	FAZ	NÃO FAZ
Interface (TUI)	TUI.java	Exibe menus, lê teclado gerencia sessão	Acessa banco, faz cálculos de negócio
Serviços	ClienteService PratoService PedidoService AvaliacaoService	Executa SQL, valida regras de negócio	Imprime no console
Modelos	Cliente, Prato Pedido, Avaliacao	Guarda dados (campos + getters)	Acessa banco ou console
Banco	Database.java	Abre conexão SQLite, cria tabelas	Executa queries de negócio

Diagram

O padrão utilizado é o **Service Layer** combinado com o padrão **Singleton** na classe Database garantindo que haja apenas uma conexão com o banco durante toda a execução do programa.

Banco de Dados

5 tabelas SQLite com relacionamentos

Todos os dados são persistidos em um único arquivo SQLite (data/restaurante.db), criado automaticamente na primeira execução:

Tabela	Colunas Principais	Descrição
clientes	id, nome, data_entrada	Clientes cadastrados. Login por nome (case-insensitive).
pratos	id, nome, preco, tempo_minutos	Cardápio. Semeado automaticamente com 8 pratos no 1º uso.
pedidos	id, id_cliente (FK), inicio, fim, entregue	Cabeçalho do pedido. fim = NULL indica pedido em aberto.
pedido_pratos	id_pedido (FK), id_prato (FK), posicao	Relação N:N entre pedidos e pratos. Posição garante a ordem.
avaliacoes	id, id_cliente (FK), nome_cliente, nota, comentario, data	Avaliações de 0 a 5 com comentário e data/hora.

Conclusão do Grupo

O que aprendemos e entregamos

O desenvolvimento do RestaurARTE representou a aplicação prática e integrada dos principais conteúdos da disciplina. O grupo concluiu o projeto com todas as funcionalidades propostas implementadas e funcionando:

O que foi entregue

- ▲ Sistema TUI completamente funcional com 8 menus navegáveis.
- ▲ Banco de dados SQLite com 5 tabelas relacionadas e integridade referencial.
- ▲ Login e cadastro unificados por nome sem senha com exibição de perfil.
- ▲ Sistema de avaliações com nota 0-5, comentário e cálculo de média via SQL AVG().
- ▲ Persistência total: nenhum dado perdido entre execuções do programa.
- ▲ Arquitetura em camadas com padrões Singleton e Service Layer aplicados.
- ▲ Prevenção de SQL Injection em todas as queries via PreparedStatement.
- ▲ Código documentado, organizado por pacotes e versionado no GitHub.

Principais aprendizados do grupo:

- ▲ A importância de separar responsabilidades: o TUI não acessa o banco, os serviços não imprimem no console, os modelos não fazem nada além de guardar dados.
- ▲ A diferença prática entre persistência em arquivo de texto e em banco de dados relacional tanto em robustez quanto em expressividade das consultas.
- ▲ O valor dos padrões de projeto: o Singleton evitou múltiplas conexões ao banco; o Service Layer centralizou a lógica e facilitou testes e manutenção.
- ▲ Como trabalhar com Git de forma colaborativa, registrando o progresso incremental do desenvolvimento em commits descritivos.